ADR-001: Desain Struktur Class Domain Akademik dan Relasi Entitas

- **Status:** Diterima (*Accepted*)
- **Tanggal:** 2026-11-01
- **Penyusun:** Arsitek Sistem (*System Architect*)
- **Konteks Sistem:** Platform Akademik Kampus (*Academic Information System / SEVIMA-Mini*)

---

1. Konteks & Latar Belakang (Context)

Sistem Informasi Akademik **SEVIMA-Mini** memerlukan fondasi pemodelan berorientasi objek yang kuat dan modular untuk mengelola keseluruhan siklus hidup civitas akademika:

- Berbagai peran pengguna (`Mahasiswa` , `Dosen` , `Admin` , `CalonMahasiswa`).
- Penyelenggaraan mata kuliah (`Course`) beserta dependensi kurikulum (`prasyarat`).
- Transaksi pengajuan rencana studi mahasiswa (`KRS`) yang menaungi registrasi mata kuliah (`Enrollment`).
- Penjadwalan alokasi ruang dan waktu perkuliahan (`Schedule`) serta manajemen kapasitas kelas yang aman dari konkurensi.
- Penilaian capaian belajar mahasiswa (`Grade`) dan formula evaluasi huruf mutu.
- Transaksi kewajiban finansial per semester (`Payment`).
- Lapisan orkestrasi bisnis (*Service Layer*) dan aturan validasi modular (*Strategy Pattern*).

Tantangan arsitektur yang diatasi meliputi:

1. Menghindari kopling ketat (*tight coupling*) dan pembentukan *God Class* pada entitas `Mahasiswa` maupun `Enrollment` .
2. Mematuhi prinsip **SOLID** secara komprehensif agar sistem mudah diuji (*unit testable*), mudah diekstensi, dan siap diimplementasikan ke kode produksi.
3. Membedakan secara presisi kepemilikan data melalui relasi **Composition** vs **Aggregation**.
4. Menjamin keamanan konkurensi alokasi kuota kelas saat periode beban puncak (*War KRS*).

---

2. Keputusan Arsitektur & Struktur Class (Architectural Decision)

2.1 Struktur Class & Tanggung Jawab (Domain Core & Services)

Entitas / Interface	Tanggung Jawab Tunggal (*Single Responsibility*)	Rationale & Pola Desain
`User` (Abstract)	Menangani identitas dasar, kredensial autentikasi, dan audit waktu.	Basis polimorfisme seluruh pengguna. Mengimplementasikan `IAuthenticatable`.
`Mahasiswa`	Mengelola profil akademik, semester, IPK, relasi Dosen PA, serta status pengisian KRS/KHS.	Turunan `User`. Mengimplementasikan `IEnrollmentObserver` untuk merespons pembaruan status KRS dan nilai.
`Dosen`	Mengelola profil pengajar, NIDN, input nilai, jadwal ajar, serta validasi KRS mahasiswa bimbingan.	Turunan `User`. Berperan sebagai Dosen Pengajar dan Dosen

		Pembimbing Akademik (`dosenPA`). Implementasi `IEnrollmentObserver`.
`Admin`	Manajemen akses user, kurikulum mata kuliah, dan verifikasi pembayaran.	Turunan `User` dengan wewenang operasional institusional.
`CalonMahasiswa`	Mengelola pendaftaran admisi baru, jalur masuk, upload berkas, dan status seleksi (`pending`, `approved`, `rejected`).	Subclass `User` sebelum migrasi akun resmi menjadi `Mahasiswa`.
`Course`	Katalog mata kuliah, bobot SKS, deskripsi, dan validasi kelulusan prasyarat kurikulum.	Menjadi wadah kurikulum akademik dengan relasi *self-referential* untuk prasyarat.
`Schedule`	Alokasi hari, jam, ruangan, kuota kapasitas, deteksi bentrok, serta penguncian kuota atomik.	Dilengkapi method `reserveKapasitas()` dan `releaseKapasitas()` untuk mitigasi *Race Condition (TOCTOU)*.
`KRS`	*Aggregate Root* dokumen rencana studi per semester, menghitung total SKS, dan memegang status approval.	Memisahkan tanggung jawab paket rencana studi mahasiswa dari transaksi mata kuliah individual.
`Enrollment`	Entitas asosiatif pendaftaran mata kuliah spesifik per semester antara `Mahasiswa` dan `Course`.	Menaungi status registrasi, nilai `Grade`, delegasi evaluator mutu, dan daftar observer.
`Grade`	Komponen nilai tugas, UTS, UAS, kalkulasi nilai akhir, dan huruf mutu.	Komponen nilai yang terikat hidup-matinya (*composition*) pada satu `Enrollment`.
`Payment`	Mengelola tagihan biaya perkuliahan (SPP/UKT), verifikasi pelunasan, batas waktu, dan metode pembayaran.	Mengisolasi audit finansial terpisah dari domain akademik perkuliahan.
`ProgramStudi`	Master data kelembagaan program studi (kodeProdi, namaProdi, jenjang).	Entitas master institusi pendukung admisi dan kurikulum.
`NIMGenerator`	Komponen utilitas penghasil Nomor Induk Mahasiswa (NIM) resmi per prodi dan tahun masuk.	Dipisahkan dari `UserFactory` untuk mematuhi prinsip *Single Responsibility (SRP)*.
`UserFactory`	Pabrik objek kreasional polimorfik pembuat turunan `User`.	*Pure Factory Pattern*, mendelegasikan pembuatan entitas pengguna tanpa terbebani algoritma penomoran NIM.
`KRSService`	Layanan orkestrator bisnis pengisian, validasi multi-rule, dan persetujuan/penolakan KRS.	Service layer pemutus logika alur KRS, mengelola eksekusi polimorfik `ValidationRule`.

`PMBService`	Layanan orkestrator pendaftaran mahasiswa baru, verifikasi berkas, dan sinkronisasi akun.	Mengelola siklus admisi pendaftar, terhubung ke `NIMGenerator` dan `UserFactory`.
`NotificationService`	Layanan pengiriman notifikasi terpusat dan penyiaran pembaruan status KRS (*Event Dispatcher*).	Menghubungkan service layer dengan kanal pengguna dan observer tanpa kopling ketat.
`ValidationRule` **(Interface)**	Kontrak strategi aturan validasi rencana studi: `validate(krs, mahasiswa): bool`.	*Strategy Pattern* modular untuk mengevaluasi kelayakan pengisian KRS.
`PaymentValidationRule`	Aturan validasi pelunasan kewajiban pembayaran SPP semester aktif.	Implementasi konkret `ValidationRule` domain finansial.
`PrerequisiteRule`	Aturan validasi kelulusan mata kuliah prasyarat kurikulum.	Implementasi konkret `ValidationRule` domain kurikulum.
`SksLimitRule`	Aturan validasi batas beban kuota maksimal SKS (≤ 24 SKS).	Implementasi konkret `ValidationRule` beban akademik.
`ScheduleConflictRule`	Aturan validasi bentrok jadwal hari, jam, dan ruangan antar-mata kuliah terpilih.	Implementasi konkret `ValidationRule` alokasi ruang/waktu perkuliahan.
`DoubleEnrollmentRule`	Aturan validasi pencegahan pendaftaran mata kuliah duplikat dalam satu KRS.	Implementasi konkret `ValidationRule` integritas data rencana studi.

2.2 Keputusan Pemilihan Relasi (Composition vs Aggregation)

A. Composition (`*--`) — Relasi Kuat / *Lifecycle-bound*

1. **`KRS` $\rightarrow$ `Enrollment`** :
   - **Alasan:** Rencana studi mahasiswa (`KRS`) menaungi daftar pendaftaran mata kuliah (`Enrollment`). Penghapusan draft KRS secara otomatis membatalkan seluruh pendaftaran mata kuliah di dalamnya.
2. **`Enrollment` $\rightarrow$ `Grade`** :
   - **Alasan:** Nilai akademik (`Grade`) tidak memiliki eksistensi mandiri tanpa adanya transaksi pendaftaran kuliah (`Enrollment`). Record nilai ikut terhapus (*cascade delete*) jika pendaftaran dibatalkan.
3. **`Course` $\rightarrow$ `Schedule`** :
   - **Alasan:** Alokasi jadwal ruang dan waktu secara semantik dimiliki oleh sesi mata kuliah yang ditawarkan pada kurikulum semester berjalan.
4. **`Mahasiswa` $\rightarrow$ `KRS`** :
   - **Alasan:** Dokumen KRS per semester dimiliki secara eksklusif oleh mahasiswa yang bersangkutan.

B. Aggregation (`o--`) — Relasi Lemah / *Independent Lifecycle*

1. **`Mahasiswa` $\rightarrow$ `Payment`** :

- **Alasan:** Riwayat pembayaran memiliki siklus hidup audit keuangan independen; pembatalan/revisi KRS tidak menghapus catatan transaksi finansial mahasiswa.

2. `Course` **$\rightarrow$** `Course` **(Prasyarat Kurikulum)**:
   - **Alasan:** Mata kuliah prasyarat tetap berdiri sendiri di basis data kurikulum meskipun mata kuliah tingkat lanjutannya direvisi atau dinonaktifkan.
3. `Course` **$\rightarrow$** `Enrollment` :
   - **Alasan:** Mata kuliah tetap tercantum di katalog program studi meskipun pada semester berjalan belum ada mahasiswa yang mendaftar.
4. `Dosen` **$\rightarrow$** `Course` :
   - **Alasan:** Penugasan dosen pengajar bersifat dinamis; pergantian penugasan dosen tidak menghapus mata kuliah.
5. `Mahasiswa` **$\rightarrow$** `Dosen` **(** `dosenPA` **)**:
   - **Alasan:** Relasi perwalian akademik menghubungkan mahasiswa dengan dosen pembimbing akademiknya secara independen.

---

3. Penerapan Prinsip SOLID

- **Single Responsibility Principle (SRP):**
  - `Schedule` bertanggung jawab atas ruang, waktu, dan reservasi kapasitas atomik.
  - `Enrollment` murni mengontrol status transaksi registrasi mata kuliah; tanggung jawab validasi pembayaran dipindahkan ke `PaymentValidationRule` .
  - `NIMGenerator` memikul tanggung jawab algoritma penomoran NIM, terpisah dari `UserFactory` yang fokus pada instansiasi objek `User` .
  - `KRSService` dan `PMBService` memisahkan logika orkestrasi bisnis dari entitas data pasif.
- **Open/Closed Principle (OCP):**
  - Penambahan aturan validasi KRS baru cukup mengimplementasikan interface `ValidationRule` tanpa perlu mengubah class `Enrollment` maupun `KRSService` .
  - Penambahan skema evaluasi nilai baru diakomodasi via `IGradeEvaluator` .
- **Liskov Substitution Principle (LSP):**
  - Setiap subclass dari `User` (`Mahasiswa` , `Dosen` , `Admin` , `CalonMahasiswa`) dapat disubstitusikan ke dalam kontrak autentikasi `IAuthenticatable` .
  - Seluruh concrete rules (`PrerequisiteRule` , `SksLimitRule` , dll.) dapat saling menggantikan dalam koleksi polimorfik `List<ValidationRule>` .
- **Interface Segregation Principle (ISP):**
  - Antarmuka dipecah spesifik dan kohesif: `IAuthenticatable` (akses login), `IGradeEvaluator` (kalkulasi nilai), `IEnrollmentObserver` (reaksi event perkuliahan), dan `ValidationRule` (evaluasi KRS).
- **Dependency Inversion Principle (DIP):**
  - `KRSService` bergantung pada abstraksi `ValidationRule` daripada implementasi aturan konkret.
  - `Enrollment` bergantung pada abstraksi `IGradeEvaluator` dan `IEnrollmentObserver` .

---

4. Design Pattern yang Diterapkan

4.1 Strategy Pattern: Validasi Rencana Studi (`ValidationRule`)

- **Masalah:** Aturan validasi pengisian KRS sangat dinamis dan berpotensi terus bertambah (syarat SPP, prasyarat mata kuliah, batas kuota SKS, deteksi bentrok jadwal, pencegahan duplikasi pendaftaran). Menyatukan seluruh aturan ini di dalam satu method menghasilkan *spaghetti code* yang melanggar SRP dan OCP.
- **Solusi:** Diterapkan *Strategy Pattern* dengan interface `ValidationRule` yang memiliki method `validate(krs: KRS, mahasiswa: Mahasiswa): bool` . Aturan-aturan konkret dipecah menjadi 5 class strategi independen.
- **Manfaat:** `KRSService` dapat mengeksekusi rangkaian validasi polimorfik melalui perulangan loop tanpa perlu mengetahui detail internal dari masing-masing aturan.

4.2 Factory Pattern & Generator Separation (`UserFactory` & `NIMGenerator`)

- **Masalah:** Pembuatan akun civitas memerlukan instansiasi polimorfik, sementara pembentukan NIM resmi mahasiswa baru memerlukan algoritma berbasis program studi dan tahun masuk.
- **Solusi:** `UserFactory` murni menginstansiasi objek `User` berdasarkan data yang diterimanya, sementara algoritma format NIM didelegasikan ke class independen `NIMGenerator` .
- **Manfaat:** Logika pembuatan objek terisolasi, mudah diuji secara unit (*unit testable*), dan tidak melanggar SRP.

4.3 Observer Pattern (`IEnrollmentObserver`)

- **Masalah:** Pembaruan status rencana studi (pengajuan, persetujuan, penolakan) dan penerbitan nilai membutuhkan notifikasi instan kepada mahasiswa dan dosen pembimbing akademik.
- **Solusi:** `Enrollment` dan `NotificationService` mempublikasikan event perubahan status ke antarmuka `IEnrollmentObserver` . Class `Mahasiswa` dan `Dosen` mengimplementasikan antarmuka ini.
- **Manfaat:** Pemisahan total (*loose coupling*) antara transaksi bisnis rencana studi dengan kanal pengiriman pesan pengguna.

---

5. Pertimbangan Konkurensi & Hardening Arsitektur

- **Pencegahan Race Condition (*TOCTOU*) Kuota Kelas:** Pengecekan kuota pasif digantikan dengan operasi atomik `reserveKapasitas()` dan transaksi kompensasi `releaseKapasitas()` pada entitas `Schedule` . Hal ini menjamin tidak terjadi *overselling* kuota kelas saat ribuan mahasiswa berebut mata kuliah secara simultan (*War KRS*).
- **Standardisasi Nomenklatur Bahasa Indonesia:** Seluruh asosiasi peran dan relasi struktural distandarisasi menggunakan Bahasa Indonesia yang konsisten (`memiliki` , `membayar` , `memuat` , `mengikuti jadwal` , `dosen PA` , `menjalankan aturan`).

---

6. Riwayat Perubahan & Evolusi Class Diagram (Architecture Evolution)

6.1 Matriks Evolusi Arsitektur Antar-Versi

Aspek Arsitektur	Baseline v1.0	Revisi v1.1 (Financial)	Revisi v2.0 (SOLID & Services)	Arsitektur Final v2. (Hardening)

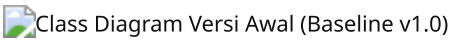
Entitas Finansial	Belum ada	Ditambahkan class `Payment`	`Payment` terintegrasi	`PaymentValidationRu` memvalidasi via Strategy
Validasi KRS	Hardcoded di `Enrollment`	Ditambahkan validasi tagihan di `Enrollment`	Polimorfik via `ValidationRule` (4 rules)	5 Strategy Rules (termasuk `DoubleEnrollmentRul`
Agregat Rencana Studi	`Enrollment` langsung ke `Mahasiswa`	`Enrollment` langsung ke `Mahasiswa`	Ditambahkan entitas agregat `KRS`	`KRS` agregat root penuh dengan statu approval
Service Layer	Tidak ada (logika di entitas)	Tidak ada	`KRSService`, `PMBService`, `NotificationService`	Service layer lengka terhubung ke Notification
Penomoran NIM	Logika bercampur	Logika bercampur	`UserFactory` mengkomposisi `NIMGenerator`	`NIMGenerator` terpisa mandiri dari `UserFactory` (SRP)
Peran Pembimbing	Belum ada	Belum ada	`dosenWali` pada `Mahasiswa`	Standardisasi istilah resmi `dosenPA` (Dose PA)
Kontrol Konkurensi	`Schedule.kapasitas` pasif	`Schedule.kapasitas` pasif	`Schedule.kapasitas` pasif	`reserveKapasitas()` & `releaseKapasitas()` atomik

---

6.2 Dokumentasi Visual Perbandingan Evolusi Diagram

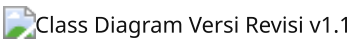
A. Versi Awal (Baseline v1.0 - Struktur Inti Akademik)

Diagram awal memodelkan relasi dasar tanpa lapisan finansial, agregat rencana studi, maupun service layer.


Class Diagram Versi Awal (Baseline v1.0)

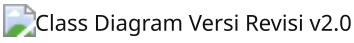
B. Versi Revisi v1.1 (Penambahan Entitas Payment)

Ditambahkan entitas `Payment` untuk mengakomodasi prasyarat pelunasan SPP sebelum rencana studi dapat diisi.


Class Diagram Versi Revisi v1.1

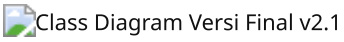
C. Versi Revisi v2.0 (Service Layer & Strategy Pattern Validation)

Restrukturisasi besar: pemisahan Service Layer (`KRSService` , `PMBService`), pembentukan interface `ValidationRule` , agregat `KRS` , dan entitas master `ProgramStudi` .


Class Diagram Versi Revisi v2.0

D. Versi Final v2.1 (Penyempurnaan Asosiasi Bahasa Indonesia, Dosen PA & Hardening)

Standardisasi penuh bahasa Indonesia, pemisahan `NIMGenerator`, penambahan `DoubleEnrollmentRule`, serta proteksi konkurensi kapasitas kelas.


Class Diagram Versi Final v2.1

---

7. Referensi Berkas Terkait

- Diagram UML Class Mermaid Utama: `diagrams/class-diagram.mmd`
- Vektor Diagram SVG: `docs/adr/diagram.svg`
- Screenshot Baseline v1.0: `docs/adr/images/class-diagram-v1-baseline.png`
- Screenshot Revisi v1.1: `docs/adr/images/class-diagram-v2-payment.png`
- Screenshot Revisi v2.0: `docs/adr/images/class-diagram-v3-update-problem.png`
- Screenshot Final v2.1: `docs/adr/images/class-diagram-v4-changes-name-association.png`